MongoDB Shell (mongosh) Reference

CRUD Operations and Aggregation — Command Documentation

1. Database and Collection Commands

Command	Description
`show dbs`	List all databases.
`use shopDB`	Switch to a database (created when data is first inserted).
`db`	Show the current database.
`show collections`	List collections in the current database.
`db.createCollection('products')`	Create a collection explicitly.
`db.products.drop()`	Delete a collection.
`db.dropDatabase()`	Delete the current database.

2. Create (Insert)

Command	Description
`db.col.insertOne(doc)`	Insert a single document.
`db.col.insertMany([doc1, doc2])`	Insert multiple documents.

```
db.students.insertOne({ name: "Asha", age: 20, dept: "CSE", marks: 85 })

db.students.insertMany([
  { name: "Ravi",  age: 21, dept: "ECE", marks: 72 },
  { name: "Meena", age: 19, dept: "CSE", marks: 91 }
])
```

Note: MongoDB adds a unique **_id** (ObjectId) automatically if it is not supplied.

3. Read (Query)

Command	Description
`db.col.find()`	Return all documents.
`db.col.find(filter)`	Return documents matching the filter.
`db.col.find(filter, projection)`	Return only selected fields.
`db.col.findOne(filter)`	Return the first matching document.
`db.col.countDocuments(filter)`	Count matching documents.
`db.col.distinct('field')`	List unique values of a field.
`.pretty()`	Format output (default in mongosh).

```
db.students.find({ dept: "CSE" })
db.students.find({ dept: "CSE" }, { name: 1, marks: 1, _id: 0 })   // projection
db.students.findOne({ name: "Asha" })
db.students.countDocuments({ marks: { $gt: 80 } })
db.students.distinct("dept")
```

Projection: 1 = include, 0 = exclude. Inclusion and exclusion cannot be mixed, except for **_id**.

3.1 Cursor Modifiers

Method	Description
`.sort({ field: 1 })`	Sort ascending (1) or descending (-1).
`.limit(n)`	Return at most n documents.
`.skip(n)`	Skip the first n documents.
`.count()`	Deprecated; use countDocuments().

```
db.students.find().sort({ marks: -1 }).skip(5).limit(5)
```

4. Query Operators

4.1 Comparison Operators

Operator	Meaning	Example
`$eq`	Equal to	{ age: { $eq: 20 } }
`$ne`	Not equal to	{ age: { $ne: 20 } }
`$gt`	Greater than	{ marks: { $gt: 80 } }
`$gte`	Greater than or equal	{ marks: { $gte: 80 } }
`$lt`	Less than	{ age: { $lt: 21 } }
`$lte`	Less than or equal	{ age: { $lte: 21 } }
`$in`	Matches any value in array	{ dept: { $in: ["CSE","ECE"] } }
`$nin`	Matches none in array	{ dept: { $nin: ["MECH"] } }

4.2 Logical Operators

Operator	Meaning	Example
`$and`	All conditions true	{ $and: [{ age: { $gt: 18 } }, { dept: "CSE" }] }
`$or`	At least one true	{ $or: [{ dept: "CSE" }, { marks: { $gt: 90 } }] }
`$nor`	None are true	{ $nor: [{ dept: "CSE" }, { age: 19 }] }
`$not`	Negates an expression	{ marks: { $not: { $gt: 50 } } }

Multiple fields in one filter are combined with an implicit AND: `{ dept: "CSE", age: 20 }`

4.3 Element and Evaluation Operators

Operator	Meaning	Example
`$exists`	Field is present	{ email: { $exists: true } }
`$type`	Field is of BSON type	{ age: { $type: "number" } }
`$regex`	Pattern match	{ name: { $regex: "^A", $options: "i" } }
`$expr`	Compare fields / use expressions	{ $expr: { $gt: ["$marks", "$age"] } }

4.4 Array Operators

Operator	Meaning	Example
`$all`	Array contains all listed values	{ tags: { $all: ["a","b"] } }
`$size`	Array has exact length	{ tags: { $size: 3 } }
`$elemMatch`	An element meets all conditions	{ scores: { $elemMatch: { $gt: 80, $lt: 90 } } }

Embedded documents and arrays are queried using dot notation: `{ "address.city": "Chennai" }`, `{ "scores.0": 90 }`

5. Update

Command	Description
`db.col.updateOne(filter, update)`	Update the first matching document.
`db.col.updateMany(filter, update)`	Update all matching documents.
`db.col.replaceOne(filter, doc)`	Replace the entire document (except _id).
`db.col.findOneAndUpdate(filter, update)`	Update and return the document.

```
db.students.updateOne({ name: "Asha" }, { $set: { marks: 90 } })
db.students.updateMany({ dept: "CSE" }, { $inc: { marks: 5 } })
db.students.updateOne({ name: "Zed" }, { $set: { age: 22 } }, { upsert: true })
```

upsert: true inserts a new document if none matches.

5.1 Update Operators

Operator	Meaning	Example
`$set`	Set or add a field value	{ $set: { age: 21 } }
`$unset`	Remove a field	{ $unset: { age: "" } }
`$inc`	Increment (or decrement) a number	{ $inc: { marks: 5 } }
`$mul`	Multiply a number	{ $mul: { marks: 1.1 } }
`$rename`	Rename a field	{ $rename: { dept: "department" } }
`$min / $max`	Update only if new value is lower / higher	{ $max: { marks: 95 } }
`$push`	Append to an array	{ $push: { tags: "new" } }
`$addToSet`	Append only if not present	{ $addToSet: { tags: "new" } }
`$pull`	Remove matching array values	{ $pull: { tags: "old" } }
`$pop`	Remove first (-1) or last (1) element	{ $pop: { tags: 1 } }
`$currentDate`	Set to current date	{ $currentDate: { updated: true } }

6. Delete

Command	Description
`db.col.deleteOne(filter)`	Delete the first matching document.
`db.col.deleteMany(filter)`	Delete all matching documents.
`db.col.deleteMany({})`	Delete all documents (collection remains).
`db.col.findOneAndDelete(filter)`	Delete and return the document.

```
db.students.deleteOne({ name: "Ravi" })
db.students.deleteMany({ marks: { $lt: 40 } })
```

7. Indexes (Brief)

Command	Description
`db.col.createIndex({ field: 1 })`	Create an ascending index.

Command	Description
`db.col.getIndexes()`	List indexes.
`db.col.dropIndex('field_1')`	Remove an index.
`db.col.find(filter).explain('executionStats')`	Show the query execution plan.

8. Aggregation

An aggregation **pipeline** passes documents through ordered **stages**; the output of each stage is the input of the next.

```
db.collection.aggregate([ { stage1 }, { stage2 }, ... ])
```

8.1 Common Pipeline Stages

Stage	Purpose
`$match`	Filter documents (same syntax as find). Place early for efficiency.
`$group`	Group by _id and apply accumulators.
`$project`	Include, exclude, or compute fields.
`$sort`	Order documents (1 / -1).
`$limit`	Keep the first n documents.
`$skip`	Skip the first n documents.
`$count`	Return the number of documents as a named field.
`$unwind`	Deconstruct an array into one document per element.
`$lookup`	Left outer join with another collection.
`$addFields / $set`	Add new computed fields.
`$out / $merge`	Write results to a collection.

8.2 $group Accumulators

Accumulator	Description
`$sum`	Total (use 1 to count documents).
`$avg`	Average value.
`$min / $max`	Smallest / largest value.
`$first / $last`	First / last value in group (use after $sort).
`$push`	Collect values into an array.
`$addToSet`	Collect unique values into an array.

8.3 Examples

```
// Average marks per department
db.students.aggregate([
  { $group: { _id: "$dept", avgMarks: { $avg: "$marks" }, total: { $sum: 1 } } }
])

// Filter, group, sort
db.students.aggregate([
  { $match: { marks: { $gte: 50 } } },
  { $group: { _id: "$dept", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])

// Project with computed field
db.students.aggregate([
  { $project: { _id: 0, name: 1, percentage: { $multiply: ["$marks", 1] } } }
])

// Group all documents (single result)
db.students.aggregate([
  { $group: { _id: null, maxMarks: { $max: "$marks" }, minMarks: { $min: "$marks" } } }
])

// Unwind an array
db.students.aggregate([ { $unwind: "$skills" }, { $group: { _id: "$skills", n: { $sum: 1 } } } ])

// Join: orders with customers
db.orders.aggregate([
  { $lookup: { from: "customers", localField: "custId",
               foreignField: "_id", as: "customer" } }
])

// Count after filtering
db.students.aggregate([ { $match: { dept: "CSE" } }, { $count: "cseStudents" } ])
```

8.4 Frequently Used Expression Operators

Operator	Use
`$add, $subtract, $multiply, $divide`	Arithmetic.
`$concat, $toUpper, $toLower, $substr`	String handling.
`$cond, $ifNull`	Conditional logic: { $cond: [cond, then, else] }.
`$year, $month, $dayOfMonth`	Date extraction.
`$size, $arrayElemAt`	Array length / element access.

Field references inside aggregation expressions are prefixed with **$** (e.g., `"$marks"`).

9. Quick Comparison: find() vs aggregate()

Method	Best used for
`find()`	Simple filtering, projection, sorting, paging.
`aggregate()`	Grouping, calculations, joins, reshaping data in multiple stages.